

Contents

guide Fast Inference from Transformers via Speculative Decoding	1
0. 先给你一个抓手	1
1. 当时的语境: 为什么这篇论文重要	2
2. 论文地图	2
3. 先把符号讲成人话	3
4. Standardized Sampling: 为什么先统一成概率分布	3
5. 单 token speculative sampling	4
6. 多 token Algorithm 1	4
7. 张量和系统图	5
8. 接受率 alpha 怎么理解	6
9. 期望生成 token 数	6
10. speedup 和 operation trade-off	7
11. 怎样选 approximation model	7
12. 实验证据链	8
12.1 Table 2: T5-XXL 的实测 walltime	8
12.2 Table 3: alpha 是否只在 T5 上好看	9
12.3 Table 4: 理论预测和实测对得上吗	9
12.4 Figure 1: 为什么可视化很有教学价值	9
13. 和本仓库代码的连接	10
14. 这篇论文没有证明什么	11
15. 和后续工作的关系	11
16. 用 AI agent 学这篇论文的正确方式	11
17. 读完后的闭卷复述模板	12

guide_Fast Inference from Transformers via Speculative Decoding

原论文: Fast Inference from Transformers via Speculative Decoding
作者: Yaniv Leviathan, Matan Kalman, Yossi Matias
会议版本: ICML 2023
本地原文 PDF: [learning/speculative-decoding/paper/01_speculative_decoding.pdf](#)
本地导读 PDF: [learning/speculative-decoding/paper/guide_01_speculative_decoding.pdf](#)
本地核心代码: [learning/speculative-decoding/src/classic_spec_decode.py](#)
本地机制实验: [learning/speculative-decoding/src/speculative_original_minimal.py](#)

0. 先给你一个抓手

Speculative decoding 要解决的是自回归解码的串行瓶颈。

普通 decoding:

```
target model runs once -> produce token 1
target model runs once -> produce token 2
target model runs once -> produce token 3
...
```

生成 K 个 token 需要 K 次串行 target forward。模型越大, 单步越贵; 而且很多生成场景 batch size 小, 延迟敏感, 用户等的是第一个字、下一句话、下一段代码。

Speculative decoding 的想法:

```
draft model guesses k tokens cheaply
target model verifies k+1 positions in one parallel pass
accept a prefix of the draft
if rejection happens, sample a corrected token
if all accepted, sample one bonus target token
```

它最重要的性质不是“快”，而是：

```
the final token distribution is exactly the target model distribution
```

如果你只把它理解成“小模型帮大模型猜”，还没有读懂。真正的核心是：小模型可以猜，但大模型用一个 rejection-sampling-like 规则来保证输出分布不变。

1. 当时的语境：为什么这篇论文重要

大模型能力强，但自回归推理天然串行。论文开头指出，生成 k 个 token 需要 k 次模型运行。GPT-3、LaMDA、PaLM、T5-XXL 这类模型的单步推理已经很贵，串行生成会把延迟放大。

已有加速方向大致有几类：

- 蒸馏：用小模型近似大模型，但输出分布会变。
- 稀疏化、量化、架构改造：可以降成本，但通常需要改模型或训练流程。
- early exit 或 adaptive computation：简单 token 少算一些，但要改结构，且常常不保证和原模型输出完全一致。
- blockwise parallel decoding：可并行猜多个 token，但多用于 greedy 或需要训练特殊组件。

这篇论文的定位很干净：

- 不改 target model 架构。
- 不重新训练 target model。
- 可以使用现成的小模型作为 approximation model。
- 对 stochastic sampling 也保持输出分布不变。
- 在内存带宽和通信是瓶颈、算力仍有富余时，用更高并发换更低 walltime。

它把处理器里的 speculative execution 类比到语言模型解码。CPU 分支预测会先猜一条分支并执行，如果猜错再修正；这里小模型先猜 token，大模型验证，猜错就用严格概率规则修正。

2. 论文地图

建议按这个顺序读：

- Section 1: 背景，说明自回归串行解码慢，以及为什么想要不改输出分布的加速。
- Section 2.1: Overview，定义 target model M_p 和 approximation model M_q 。
- Section 2.2: Standardized Sampling，把 argmax、top-k、nucleus、temperature 都视为 adjusted probability distribution。
- Section 2.3: Speculative Sampling，给出单 token 接受/拒绝规则。
- Algorithm 1: SpeculativeDecodingStep，多 token 版本。
- Section 3.1 到 3.5: 分析接受率、期望生成 token 数、walltime speedup、operation trade-off、怎样选 γ 。
- Section 3.6: approximation model 怎么选。
- Section 4: T5-XXL 实验和各类模型的 empirical α 。
- Appendix A.1: 正确性证明。这是最该反复读的部分。
- Appendix A.3: 理论预测和实测 runtime 对比。
- Appendix A.4 和 A.5: beam search 和 lenience 的扩展讨论。

图表抓重点:

- Figure 1: 一个 38-token 例子, target 只串行跑了 9 次; 绿色是被接受的 draft, 红色是拒绝, 蓝色是修正。
- Figure 2: 不同 γ 下, α 越高, 单次 iteration 期望生成 token 越多。
- Figure 3: 给定 α 和 draft cost c , 最佳 γ 怎么变。
- Figure 4: speedup 和 arithmetic operations 的权衡。
- Figure 5: encoder-decoder T5 trace, 直观看 draft decoder 和 target decoder 的并发关系。
- Table 2: T5-XXL 上 2X 到 3X 的实际 walltime 加速。
- Table 3: 各任务、各 approximation model 的 empirical α 。
- Table 4: 理论 speedup 和实测 speedup 的对照。

3. 先把符号讲成人话

论文用两个模型:

M_p : target model

大模型, 输出我们必须严格保持的分布 p

M_q : approximation model

小模型或便宜启发式, 输出 draft 分布 q

给定当前 prefix:

$p(x)$ = target model says next token is x

$q(x)$ = draft model says next token is x

γ :

一次 speculative iteration 中, draft model 先猜多少个 token。

α :

平均接受率。粗略说, q 越接近 p , α 越高。

c :

draft model 跑一步的 walltime 成本除以 target model 跑一步的 walltime 成本。 c 越小, 小模型越便宜。

三个量的关系:

high α :

draft guesses are often accepted

low c :

draft guesses are cheap

good γ :

large enough to exploit acceptance

small enough to avoid wasting draft work

4. Standardized Sampling: 为什么先统一成概率分布

论文 Section 2.2 很容易被跳过, 但它其实很关键。

实际生成里有很多 sampling mode:

- greedy 或 argmax
- temperature
- top-k
- nucleus sampling

这些看起来不一样, 但都可以看成先把 logits 变成一个 adjusted probability distribution, 然后从这个分布采样。比如 greedy 可以看成把最大 token 的概率设为 1, 其它 token 设为 0。

因此论文只需要讨论:

sample x from $p(x)$

而不是为每种 decoding mode 单独写算法。这个抽象很重要, 因为 speculative decoding 的正确性证明是分布级的, 不是 logits 级的。

5. 单 token speculative sampling

先看一个 token。目标是从 p 采样, 但我们先从便宜的 q 采一个 token:

$x \sim q$

如果这个 token 在 target 下并不比 draft 更低概率, 也就是 $q(x) \leq p(x)$, 那么直接接受它。

如果 $q(x) > p(x)$, 说明 draft 对这个 token 给得太乐观。此时按概率拒绝:

accept probability = $p(x) / q(x)$

reject probability = $1 - p(x) / q(x)$

如果拒绝, 不是直接从 p 重采。论文的关键是从 residual distribution 采:

$\text{residual}(x) = \text{norm}(\max(0, p(x) - q(x)))$

意思是: 只从 target 比 draft 缺少的概率质量里补回来。

单 token 的输出分布可以写成:

accepted part:

$$q(x) * \min(1, p(x) / q(x)) = \min(p(x), q(x))$$

rejected correction:

$$p(x) - \min(p(x), q(x))$$

sum:

$$\min(p(x), q(x)) + p(x) - \min(p(x), q(x)) = p(x)$$

这就是 Appendix A.1 的核心证明。你可以不记论文里的所有符号, 但必须记住这个分解:

target distribution

= overlap accepted from q

+ target surplus restored by residual sampling

6. 多 token Algorithm 1

Algorithm 1 是单 token 规则的多 token 版本。

流程:

Input:

```
target model Mp
draft model Mq
prefix
gamma
```

Step 1:

```
use Mq autoregressively to draft gamma tokens
```

Step 2:

```
run Mp in parallel on:
  prefix
  prefix + draft token 1
  prefix + draft tokens 1..2
  ...
  prefix + draft tokens 1..gamma
```

Step 3:

```
scan drafted tokens from left to right
accept while random test passes
stop at first rejection
```

Step 4:

```
if rejection happens:
  sample corrected token from residual distribution

if all gamma draft tokens are accepted:
  sample one bonus token from p at position gamma+1
```

为什么 target 要算 $\gamma + 1$ 个分布?

如果前 γ 个 draft 全部接受, 算法还要额外从 target 采一个 bonus token。这样一次 target 并行验证最多能产出 $\gamma + 1$ 个 token。

为什么最坏也不会更差?

即使第一个 draft token 立刻被拒绝, 算法也会从 residual distribution 采一个修正 token。因此每次 iteration 至少产出 1 个 token。它不会比普通解码需要更多 target 串行步数, 只是可能多花 draft 计算和并发 target 计算。

7. 张量和系统图

普通自回归:

```
time 1: target(prefix) -> token a
time 2: target(prefix a) -> token b
time 3: target(prefix a b) -> token c
```

Speculative decoding:

```
draft:
  q1 = Mq(prefix)
  x1 ~ q1
```

```

q2 = Mq(prefix x1)
x2 ~ q2
q3 = Mq(prefix x1 x2)
x3 ~ q3

```

```

target parallel verify:
  p1 = Mp(prefix)
  p2 = Mp(prefix x1)
  p3 = Mp(prefix x1 x2)
  p4 = Mp(prefix x1 x2 x3)

```

```

accept scan:
  x1 accepted or corrected
  then maybe x2
  then maybe x3
  then maybe bonus from p4

```

对 Transformer 来说，这里的“并行”不是让未来 token 神奇地不依赖过去，而是因为 draft 已经给出了候选 prefix。target 可以把这些候选位置打包成一次 forward，得到每个位置上 target 对 draft token 的概率。

这也是为什么 draft model 和 target model 通常要共享 tokenizer，并且最好使用同一类 sampling standardization。不然 candidate token 和概率空间对不上。

8. 接受率 alpha 怎么理解

论文定义 prefix 下的接受率，随后把平均接受率记作 α 。直觉上：

```
alpha = sum_x min(p(x), q(x))
```

它也等于 $1 - \text{DLK}(p, q)$ ，这里 DLK 是论文定义的对称 divergence。你不必把 DLK 当成必须背的概念，重点是：

```

p and q overlap more -> alpha higher
p and q overlap less -> alpha lower

```

如果 $q = p$ ：

```

alpha = 1
all draft tokens accepted

```

如果 p 和 q 支持集完全不重合：

```

alpha = 0
draft never helps

```

论文 Table 3 的经验结论很有用：

- 几乎无参数的 unigram/bigram model 也可能有非零 α 。
- 几千万到几亿参数的小 Transformer 通常能给 0.5 到 0.9 的 α 。
- distribution 越尖锐， α 往往越高；greedy 或低温时更容易接受。

9. 期望生成 token 数

在简化假设下，接受事件独立同分布，平均接受率为 α 。一次 Algorithm 1 期望生成 token 数是：

$$E_{\text{tokens}} = 1 + \alpha + \alpha^2 + \dots + \alpha^{\gamma}$$

equivalently:
$$(1 - \alpha^{(\gamma + 1)}) / (1 - \alpha)$$

这个式子很好理解:

- 第 1 个 token 总会产生, 所以有 1。
- 第 2 个 token 需要第 1 个 draft 被接受, 所以乘 α 。
- 第 3 个 token 需要前两个 draft 都被接受, 所以乘 α^2 。
- 最多到 bonus token, 所以到 α^{γ} 。

这就是 Figure 2。 α 越高、 γ 越大, 单次 target 串行步可以产出的 token 越多。

但 γ 不是越大越好, 因为 draft 也有成本。

10. speedup 和 operation trade-off

设:

$$c = \text{one draft step walltime} / \text{one target step walltime}$$

一次 speculative iteration 的 walltime 近似为:

$$\begin{aligned} &\text{one target parallel verify} + \gamma \text{ draft steps} \\ \text{cost} &= 1 + \gamma * c \end{aligned}$$

因此理论 speedup 可写成:

$$\text{speedup} = E_{\text{tokens}} / (1 + \gamma * c)$$

这对应论文 Theorem 3.8 的直觉版本。

这里有一个很重要的工程点:

Speculative decoding 可能增加总 arithmetic operations。因为 target 在一次并行 pass 中要算 $\gamma + 1$ 个位置, draft 也要多跑 γ 步。如果系统已经算力满载, 它可能不帮忙。

它能加速的典型原因是:

large model decoding is often memory-bandwidth or communication bound
extra arithmetic concurrency is available
serial target steps are reduced

所以它和 GPTQ 有一个共同点: 都在利用推理瓶颈不纯粹是 FLOPs。GPTQ 减少权重搬运; speculative decoding 减少 target 串行次数, 用并发换延迟。

11. 怎样选 approximation model

论文 Section 3.6 讨论了几类 M_q 。

最常见选择:

- 同架构小模型。
- 同 tokenizer。
- 同 sampling standardization。
- 参数量比 target 小约两个数量级时, 常能平衡 α 和 c 。

太小的问题:

c is tiny
but alpha may be low
many guesses rejected

太大的问题:

alpha may be high
but c is no longer small
draft cost eats the gain

论文里 T5-small 在 T5-XXL 实验中通常最合适。T5-base 和 T5-large 的 alpha 更高, 但它们也更贵, 所以 walltime speedup 反而低。

论文还提到几种特殊 approximation:

- n-gram: 几乎零成本, alpha 小但也能有一点加速。
- copying heuristic: 在摘要、对话、代码修改这类重复场景可能有效。
- non-autoregressive model: 一次给出多个 token, 不用 draft loop。
- random model: 理论上也能保证正确, 只是帮助很小。

12. 实验证据链

12.1 Table 2: T5-XXL 的实测 walltime

实验设置:

- target model: T5-XXL 11B。
- tasks: English to German translation, CNN/DailyMail summarization。
- draft models: T5-small 77M, T5-base 250M, T5-large 800M。
- hardware: 单 TPU-v4。
- batch size: 1。
- decoding: temp=0 的 argmax, 以及 temp=1 的标准采样。

English to German, temp=0:

- T5-small, gamma=7, alpha=0.75, speed 3.4X。
- T5-base, gamma=7, alpha=0.80, speed 2.8X。
- T5-large, gamma=7, alpha=0.82, speed 1.7X。

English to German, temp=1:

- T5-small, gamma=7, alpha=0.62, speed 2.6X。
- T5-base, gamma=5, alpha=0.68, speed 2.4X。
- T5-large, gamma=3, alpha=0.71, speed 1.4X。

CNN/DailyMail, temp=0:

- T5-small, gamma=5, alpha=0.65, speed 3.1X。
- T5-base, gamma=5, alpha=0.73, speed 3.0X。
- T5-large, gamma=3, alpha=0.74, speed 2.2X。

CNN/DailyMail, temp=1:

- T5-small, gamma=5, alpha=0.53, speed 2.3X。
- T5-base, gamma=3, alpha=0.55, speed 2.2X。
- T5-large, gamma=3, alpha=0.56, speed 1.7X。

这张表的读法:

T5-large 的 alpha 往往高于 T5-small, 但它太贵。真正的最优不是 “draft 越准越好”, 而是 α / c 的平衡。

12.2 Table 3: alpha 是否只在 T5 上好看

Table 3 测了更多 target/draft 组合。

GPT-like 97M:

- unigram 和 bigram 在 temp=0 或 temp=1 下 alpha 只有约 0.03 到 0.05。
- GPT-like 6M draft 在 temp=0 时 alpha 约 0.88, 在 temp=1 时约 0.89。

T5-XXL English to German:

- unigram temp=0 alpha 0.08, bigram temp=0 alpha 0.20。
- T5-small temp=0 alpha 0.75, T5-base 0.80, T5-large 0.82。
- temp=1 时, T5-small 0.62, T5-base 0.68, T5-large 0.71。

T5-XXL CNN/DailyMail:

- T5-small temp=0 alpha 0.65, temp=1 alpha 0.53。
- T5-base temp=0 alpha 0.73, temp=1 alpha 0.55。
- T5-large temp=0 alpha 0.74, temp=1 alpha 0.56。

LaMDA 137B dialog:

- LaMDA 100M temp=0 alpha 0.61, temp=1 alpha 0.57。
- LaMDA 2B temp=0 alpha 0.71, temp=1 alpha 0.71。
- LaMDA 8B temp=0 alpha 0.75, temp=1 alpha 0.74。

这说明 speculative decoding 不是只在一个 T5 设置上成立。只要 draft 分布和 target 分布有足够重叠, 就有机会。

12.3 Table 4: 理论预测和实测对得上吗

Appendix Table 4 比较 predicted speedup 和 empirical speedup。例子:

- EnDe T5-small temp=0: expected 3.2, measured 3.4。
- EnDe T5-base temp=1: expected 2.4, measured 2.4。
- CNNDM T5-large temp=1: expected 1.6, measured 1.7。

也有偏差, 例如 EnDe T5-large temp=0 expected 2.5, 但 measured 1.7。论文解释主要来自:

- speculative implementation 和 T5X baseline 的优化差异。
- beta 独立同分布假设只是近似。

这很诚实。理论公式给的是设计方向, 不是每个系统上的精确延迟预言。

12.4 Figure 1: 为什么可视化很有教学价值

Figure 1 用一个 97M target 和 6M draft 的语言模型例子展示。一个 38-token 句子, target 只串行运行了 9 次。在某一行里, target 一次运行产出了 5 个 token。

它展示了三种 token:

- accepted draft tokens: 小模型猜对, 大模型接受。

- rejected draft token: 小模型猜得太偏。
- correction token: 从 residual distribution 采样补回来。

读 Figure 1 时不要只看颜色。你要问:

At the first rejection,
why must all later draft tokens be discarded?

答案是: 后续 draft token 的条件 prefix 已经错了。自回归分布依赖完整 prefix, 一旦前面某个 token 被修正, 后面的 draft 概率就不再对应当前真实 prefix。

13. 和本仓库代码的连接

核心实现:

learning/speculative-decoding/src/classic_spec_decode.py

你要重点看:

- rejection_sample(p, q, drafted, rng): 单 token 接受/拒绝和 residual sampling。
- speculative_decode_step(...): Algorithm 1 的一次 iteration。
- run_classic_spec(...): draft gamma 个 token, target 验证 gamma+1 个位置, 更新 metrics。

我补的 paper-shaped 机制实验:

learning/speculative-decoding/src/speculative_original_minimal.py
learning/speculative-decoding/src/tests/test_speculative_original_minimal.py

它把论文证明变成可执行对象:

- overlap_mass(p, q): 计算 $\sum \min(p, q)$, 也就是 alpha。
- residual_distribution(p, q): 计算 $\text{norm}(\max(0, p - q))$ 。
- exact_one_step_output_distribution(p, q): 枚举单步输出, 检查它严格等于 p。
- expected_tokens_per_iteration(alpha, gamma): 对应 Figure 2。
- walltime_speedup(alpha, gamma, draft_cost_ratio): 对应 Theorem 3.8 的工程直觉。
- best_gamma(...): 对应 Figure 3 的选择问题。

运行:

```
.\\.venv\\Scripts\\python.exe `
  learning\\speculative-decoding\\src\\speculative_original_minimal.py

\\.venv\\Scripts\\python.exe `
  learning\\speculative-decoding\\src\\tests\\test_speculative_original_minimal.py

\\.venv\\Scripts\\python.exe -m pytest `
  learning\\speculative-decoding\\src\\tests -q
```

Capstone:

learning/speculative-decoding/src/capstone_eagle3.py

它用 synthetic task 比较 classic、Medusa、EAGLE-1、EAGLE-2。注意这不是论文 benchmark, 而是课程里的机制对比。不要拿它的 speedup 数字当真实生产结论。

14. 这篇论文没有证明什么

它没有证明:

- 所有任务都能 2X 到 3X。
- 只要小模型越大就越快。
- speculative decoding 会减少总 FLOPs。
- 没有额外并发算力时也能加速。
- beam search 已经完整解决。
- lenience 不会改变输出分布。
- 任何 tokenizer 不一致的 draft/target 都能直接套用。
- 后续 draft token 在拒绝后还能继续保留。

尤其要记住:

strict Algorithm 1:
output distribution unchanged

lenience:
may speed up more
but no longer 是完全相同分布

Appendix A.5 的 lenience 是有边界的近似策略，不是主算法结果。论文主结果使用最严格版本。

15. 和后续工作的关系

Speculative decoding 奠定了后面很多推理加速方法的共同模板:

produce candidates cheaply
verify candidates with target behavior
accept as many as possible
preserve or control output distribution

后续方法改变的是 draft 的来源:

- Medusa: 在同一个 backbone 上加多个 prediction heads。
- EAGLE: 用 feature-level autoregression 产生 draft。
- EAGLE-2: 用 dynamic tree 提高候选覆盖。
- Lookahead: 利用 n-gram 或 Jacobi-like 结构猜未来 token。
- Self-speculative decoding: 用同一个模型的浅层或跳层作为 draft。
- vLLM/SGLang/TensorRT-LLM 里的 speculative serving: 把这套思想工程化到 batch、KV cache、scheduler 和 kernel。

所以读这篇时，别急着跳到 EAGLE。先把 strict acceptance rule 和 residual distribution 读透，否则后面的树、head、feature draft 都会变成一堆名词。

16. 用 AI agent 学这篇论文的正确方式

不要让 agent 泛泛总结。你要让它检查你是否真的理解“无偏加速”。

可以这样提问:

我正在学习 Fast Inference from Transformers via Speculative Decoding。

请你一次只问我一个问题。

问题必须来自下面七类之一:

1. 为什么自回归 decoding 是串行瓶颈
2. standardized sampling 的意义
3. 单 token speculative sampling 的接受/拒绝规则
4. residual distribution 为什么能补回 target surplus
5. Algorithm 1 中 $\gamma+1$ 个 target 分布的作用
6. α 、 γ 、 c 和 speedup 的关系
7. 本仓库 classic_spec_decode.py 或 speculative_original_minimal.py 的函数映射

我回答后，请指出漏洞。

每次都要求我把答案映射到论文 section、公式、图表或本地代码函数。

最后让我用 200 字闭卷复述。

你还可以让 agent 做概率验算：

给我一个三 token 词表。

设 $p=[0.5,0.3,0.2]$ ， $q=[0.35,0.45,0.2]$ 。

请让我手算：

1. α
2. accepted part
3. residual distribution
4. final output distribution

不要直接给答案。

我算完后再指出错误。

17. 读完后的闭卷复述模板

按这个模板复述：

这篇论文的背景是：

...

普通 decoding 慢在：

...

Speculative decoding 的核心流程是：

...

它保持 target 分布不变的原因是：

...

α 表示：

...

γ 和 c 的取舍是：

...

Table 2 证明：

...

Table 3 证明：

...

它的限制是:

...

我能在本仓库看到的最小实现是:

...

如果你能不看 guide 填完这段, 并能打开 `speculative_original_minimal.py` 解释为什么 `exact_one_step_output_distribution` 等于 `target distribution`, 这篇论文就真正进脑子了。